

Leihcycle

Beschreibung

„Leihcycle“ ist ein webbasierter Fahrradverleih. Das System ermöglicht die Verwaltung einer Fahrradflotte, bei der Nutzer Fahrräder ausleihen und zurückgeben können. Nutzerprofildaten werden verwaltet, und Betreiber können den Flottenstatus über ein Live-Dashboard in Echtzeit überwachen.

Funktionale Anforderungen

Betreiber

- Fahrräder mit Eigenschaften (Modell, Typ, Standort als Koordinaten, Status) ins System einpflegen, bearbeiten, abrufen und löschen.
- Rückgabestationen mit Namen und Koordinaten anlegen, bearbeiten und löschen.
- Live-Dashboard mit aktuellem Status aller Fahrräder (verfügbar / ausgeliehen) in Echtzeit – Änderungen werden ohne Seitenreload via WebSocket übertragen.
- Nutzungsstatistiken abrufen (z.B. Anzahl Ausleihen pro Tag, beliebteste Stationen, Gesamteinnahmen).

Nutzer

- Registrieren, Profil bearbeiten und Konto löschen.
- Verfügbare Fahrräder in einem wählbaren Radius um einen angegebenen Standort suchen; Ergebnisse werden mit Entfernung und Modellangabe sortiert zurückgegeben.
- Verfügbares Fahrrad ausleihen; das System sperrt es dabei atomar gegen gleichzeitige Anfragen anderer Nutzer.
- Ausleihe durch Rückgabe an einer definierten Station beenden.
- Aktive Ausleihe sowie vollständige Ausleihhistorie einsehen.
- Automatisch berechnete Kostenabrechnung nach Rückgabe (Grundgebühr + Zeit × Minutenpreis).
- WebSocket-Benachrichtigung bei serverseitiger Bestätigung der Ausleihe oder Abschluss der Rückgabe.

System

- Logs für alle relevanten Ereignisse (Ausleihe, Rückgabe, Registrierung, Fehler).
- Tägliche Aggregation von Nutzungsstatistiken aus den Ausleih- und Rückgabedaten.
- Automatische tägliche Bereinigung (Scheduled Task) abgeschlossener Ausleihen und Logs, die älter als ein definiertes Limit sind.

Nicht-funktionale Anforderungen

Bei diesen Anforderungen wird davon ausgegangen, dass der Server auf einem handelsüblichen Laptop läuft.

- Der Server beantwortet korrekte Anfragen zu >99,9% ohne internen Fehler.
- Der Server beantwortet bei einer Rate von <50 Anfragen / Sekunde jede der Anfragen in unter 0,2 s.
- Das Server-Programm nutzt bei mehrkernigen Systemen die verfügbaren Kerne dynamisch für parallelisierbare Aufgaben.
- Gleichzeitige Ausleiheanfragen für dasselbe Fahrrad sind atomar und thread-safe, um Doppelausleihen zu verhindern.
- Status-Updates (Ausleihe / Rückgabe) werden innerhalb von maximal 3 Sekunden an alle verbundenen Clients via WebSocket übertragen.
- Der Server ist innerhalb von 15 s nach Start bereit, Anfragen entgegenzunehmen (Programm bereits mit allen Dependencies gebaut).

Verteilung

Nebenläufigkeit

- Multithreading bei der Aggregation von Nutzungsstatistiken (parallele Auswertung nach Station, Zeitraum, Nutzer).
- Multithreading bei der Abfrage und Entfernungsberechnung aller Fahrräder für die Radius-Suche (Haversine-Formel pro Fahrrad parallelisierbar).

Echtzeit-Kommunikation

- Echtzeit-Statusübertragung aller Fahrräder (verfügbar / ausgeliehen) an das Betreiber-Dashboard via WebSocket.
- Bestätigungs- und Abschluss-Benachrichtigungen für Nutzer bei Ausleihe und Rückgabe via WebSocket.

Architektur

- Client-Server-Architektur mit Spring Boot (REST API + WebSocket).

Last-Simulation & Test der nicht-funktionalen Anforderungen

Anforderung	Überprüfung
Server beantwortet korrekte Anfragen zu >99,9% ohne internen Fehler	Serverseitiges Logging unterscheidet 4xx- von 5xx-Antworten. Testclient stellt valide Anfragen, Server loggt Gesamt- und Fehleranzahl; Verhältnis wird ausgegeben.
Server beantwortet bei <50 req/s jede in unter 0,2 s	Testclient sendet Anfragen getaktet mit 50 req/s und misst per Zeitstempel die Latenz jeder Anfrage. Ausgabe: Anzahl zufriedienstellender / unzureichender Antwortzeiten.

Anforderung	Überprüfung
Mehrkernigkeit wird dynamisch genutzt	Parallelisierte Tasks loggen ihren Thread-Namen. Verhältnis genutzter zu vorhandenen Threads zeigt Kernauslastung.
Ausleiheanfragen atomar und thread-safe	Testclient startet 1.000 gleichzeitige Anfragen für dasselbe Fahrrad. Danach Status-Abfrage: Anzahl erfolgreicher Ausleihen muss genau 1 sein.
Status-Updates innerhalb von 3 s via WebSocket	Client verbindet sich per WebSocket und abonniert Updates. Zweiter Client löst Ausleihe aus; Sende- und Empfangszeitpunkt werden gestempelt; Differenz wird geloggt.
Server startet innerhalb von 15 s	Spring Boot loggt Startzeit automatisch als „Started Application in X seconds“. Überprüfung in den Startup-Logs nach jedem Build.